Kevin Küster

Zusammenfassung: Expressions

Expressions

Ausdrücke

Konstante: `X := 1;`

Arithmetische Ausdrücke: `y := (1+3)/(2*17);`

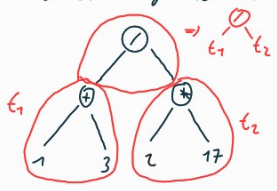
▷ Übersetzen arithmetischer Ausdrücke via Kellerspeicher (stack)

↳ Zwischenberechnung von Teilausdrücken

Operationen:

- Auf Stack legen: PUSH x
- Obersten Wert von Stack legen + entf.: POP

⇒ Post-Order Syntaxbaum



⇒ 1 3 + x 17 * /

PUSH 1
PUSH 3
ADD

⇒ ADD W I 1, I 3, 111

111: Speicherort im Stack

M1-Assembler Notation

Push: -! Ri
Pop: ! Ri +

} Verschieben des Stack-Pointers

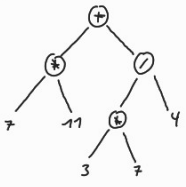


⇒ -!SP : push
!SP+ : pop

Beispiel

(7 * 11) + (3 * 7) / 4

Syntaxbaum:



Traversierung: 7 11 * 3 7 * 4 / +

↳ $t_0 := 7 * 11$
$t_1 := 3 * 7$
$t_2 := t_1 / 4$
$t_3 := t_0 + t_2$

M1-Assembly:

MUL W I 7, I 11, -!SP
MUL W I 3, I 7, -!SP
DIV W I 4, !SP+, -!SP
ADD W !SP+, !SP+, -!SP

Notwendige Speicherzugriffe

Jeder SP-Befehl benötig Speicherzugriff (Beispiel: 7)

↳ Reduzierung durch Einsatz von Registern
(Zugriff der CPU in einem Takt)

Benötigte Register - Triviale Lösung:

Anzahl Register anhand benötigter Zwischenergebnisse

↳ Beispiel: $t_0, .., t_3 \rightarrow 4$

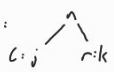
⇒ Bei komplexen Berechnungen ggf. mehr Zwischenwerte, als verfügbare Register → Müssen auf Stack gelegt werden (Register spilling)

Reduktion via Sethi-Ullman Algorithmus

(i) Alle Blätter mit arithmetischen Ausdrücken brauchen 0 oder 1 Register

- 1, falls Variable
- 0, falls Konstante

(ii) Knoten:



c,r: Linker/rechter Teilbaum
j,k: Anzahl Register

- $j = k \Rightarrow n := j + 1$
- $j \neq k \Rightarrow n := \max(j,k)$

Beispiel

`x := (a+b) * (c-d);`

Stack-Adressen:

a: -4(FP), b: -8(FP), c: -12(FP), d: -16(FP)

$t_0 := a+b$, $t_1 := c-d$, $t_2 := t_0 * t_1$

MOVE W -4(FP), R0 ; R0 = a
ADD W R0, -8(FP), -!SP ; t_0 = a+b, push

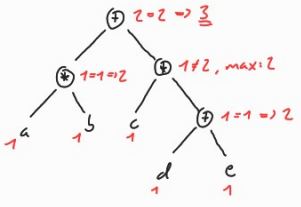
MOVE W -12(FP), R0 ; R0 = c
SUB W R0, -16(FP), -!SP ; t_1 = c-d, push

MUL W !SP+, !SP+, -!SP ; t_2 = pop * pop, push

Beispiel - Sethi-Ullman

z := (a*b) + (c * (d+e));

a, b, c, d, e: Label ≙ 1



Beispiel - Register Spilling

Benötigte Register: 3
Verfügbare Register: 2

MOVE W, d, R0 ; R0 = d
ADD W, R0, e, R0 ; R0 = d+e

MUL W, c, R0, R0 ; R0 = c * (d+e)

MOVE W, R0, -!SP ; spill → Stack: c * (d+e)

MOVE W, a, R0 ; R0 = a
MUL W, R0, b, R0 ; R0 = a * b

ADD W, R0, !SP+, R0 ; R0 = (a*b) + (c*(d+e))

⇒ Speicherzugriffe: 2, Nur Stack: 10